

Chapter

Class As The Basis Of All Computations

1. A class implements which of the following Object Oriented programming characteristic? COMP 2022

- (a) Encapsulation
- (b) Inheritance
- (c) Abstraction
- (d) Polymorphism

2. Which of the following is not a keyword? COMP 2023

- (a) Void
- (b) Byte
- (c) Object
- (d) Public

3. A variable whose value is not associated to an object and its value exists throughout the life of the program is a? SQP 2024

- (a) Local variable
- (b) Static variable
- (c) Final variable
- (d) None of these

4. What does a class encapsulate? MAIN 2024

5. What are the different set of data types available in Java? COMP 2023

6. Difference between public and private modifiers for members of a class. SQP 2022

7. Statement 1 : Composite data types in Java can combine different types of data into a single entity. **Statement**

2 : Primitive data types like int and float are examples of composite data types.

- (a) Both statements are true.
- (b) Both statements are false.
- (c) Statement 1 is true, but Statement 2 is false.
- (d) Statement 1 is false, but Statement 2 is true.

SQP 2023

8. Which of the following is NOT a feature of ObjectOriented Programming (OOP) implemented by classes in Java? SQP 2024

- (a) Abstraction
- (b) Inheritance
- (c) Encapsulation
- (d) Compilation

9. Write a program using Java, to print the real quotient obtained by dividing two integer numbers. Also, print the remainder obtained after dividing them. SQP 2023

10. Write a program using Java to print the real quotient obtained by dividing two integer numbers. Also, print the remainder obtained after dividing them. MAIN 2021

11. Write a program using Java to calculate and display the sum and difference of two integer numbers entered by the user. SQP 2022

12. Write a Java program that computes the double and half of a, where a stores 10.92. MAIN 2021

Solution

1. (a) Encapsulation

A **class** in Object-Oriented Programming **encapsulates data (attributes) and behavior (methods)** within a single unit, demonstrating the concept of **encapsulation**.

2. (c) Object

In Java, Object is a class name, not a **reserved keyword**. Keywords like void, byte, and public are reserved and cannot be used as identifiers.

3. (b) Static variable

A **static variable** belongs to the **class** rather than any specific object. Its value **persists throughout the life of the program** and is shared among all instances of the class.

4. A class encapsulates:

1. **Data (Attributes):** Properties of the objects it creates, e.g., color or speed in a Car class.
2. **Behavior (Methods):** Actions the objects can perform, e.g., accelerate() or brake().

This **bundling of data and methods** ensures objects maintain their state and behavior, following OOP principles like **encapsulation and abstraction**.

5. Java provides two main categories of data types:

1. **Primitive Data Types:** Basic data types such as int, float, double, char, boolean, byte, short, and long.
2. **Reference Data Types:** Include **objects, arrays, and classes**, which store references (memory addresses) to complex data structures.

6. Public members: Can be accessed from **any class** in **any package**.

Private members: Can be accessed **only within the class** in which they are declared.

7. (c) Statement 1 is true, but Statement 2 is false

Statement 1: True — Composite data types, such as **classes and arrays**, combine multiple related values into a single entity.

Statement 2: False — **Primitive types** like int and float are **basic data types**, not composite types.

8. (d) Compilation

Compilation is a process of translating code into bytecode, **not a feature of Object-Oriented Programming**. OOP features implemented by classes include **abstraction, inheritance, encapsulation, and polymorphism**.

9. class Real {

```
    public static void main(String[] args) {
```

```
        int num1, num2, rem;
```

```
        double q;
```

```
        num1 = 5;
```

```
        num2 = 50; q = (double) num1 / num2; // Real quotient
```

```
        rem = num1 % num2;    // Remainder
```

```
        System.out.println("The result is " + q);
```

```
        System.out.println("Remainder = " + rem);
```

```
    }
```

```
}
```

10.

```
import java.util.Scanner;

class QuotientRemainder {
    public static void main(String[]
args) {
        Scanner sc = new Scanner(System.
in);

        // Input two integers
        System.out.println("Enter the
first integer (dividend) : ");
        int dividend = sc.nextInt();
        System.out.println("Enter the
second integer (divisor) : ");
        int divisor = sc.nextInt();

        // Calculate quotient and
remainder
        int quotient = dividend /
divisor;
        int remainder = dividend %
divisor;

        // Print the results
        System.out.println("Quotient
: " + quotient);
        System.out.println("Remainder
: " + remainder);

        sc.close();
    }
}
```

11.

```
import java.util.Scanner;

class Sum Difference {
    public static void main(String[]
args) {
        Scanner sc = new Scanner(System.
in);
```

```

        // Input two integers
        System.out.println("Enter the
first number : ");
        int num1 = sc.nextInt();
        System.out.println("Enter the
second number : ");
        int num2 = sc.nextInt();

        // Calculate sum and difference
        int sum = num1 + num2;
        int difference = num1 - num2;

        // Print the results
        System.out.println("Sum : "
+ sum);
        System.out.println("Difference
: " + difference);

        sc.close();
    }
}

```

12.

```

class Compute {
    public static void main() {
        double a, d, h;
        a = 10.92;
        d = 2 * a;
        h = 1 / 2.0 * a;
        System.out.println("Double of
" + a + " = " + d);
        System.out.println("Half of
" + a + " = " + h);
    }
}

```

Variables used :

a : the given number

d : stores the double of a

h : stores the half of a